WEEK 09

The Web Server

Back-End Development: Your First Steps Behind the Curtain

Web Applications Development | Undergraduate

Cyber.SoHo Educational Hub © 2026 | Schiller International University

Today's Roadmap

- Lesson 1: What Is a Web Server?
- Lesson 2: HTTP — The Language of the Web
- Lesson 3: Static vs Dynamic Content
- Lesson 4: The Big Three — Apache, Nginx, Node.js
- Lesson 5: Ports, Listening & localhost
- Lesson 6: Build Your First Web Server
- Lesson 7: Routing — Directions Inside Your Server
- Lesson 8: Middleware — Security Guards of the Pipeline

Lesson 1 — What Is a Web Server?

- **A programme (or machine) that listens for requests and responds with content**
  - Like a restaurant kitchen that never closes

- **Two meanings of 'Web Server':**
  - Hardware: the physical computer storing files & running server software
  - Software: the programme (Apache, Nginx, Node.js) understanding HTTP

> *Key Takeaway: Every website is powered by a server — a programme waiting 24/7 to answer your browser.*

The Request → Response Cycle

Step	Pizza World	Web World
1	You call Pizza Hut	Browser sends HTTP request
2	Operator takes order	Server receives request
3	Kitchen cooks pizza	Server processes request
4	Driver delivers box	Server sends HTTP response
5	You open box and eat	Browser renders the page

Key Takeaway: All web communication is a two-step dance — request in, response out.

Lesson 2 — HTTP (HyperText Transfer Protocol)

- **A text-based protocol defining how messages travel on the Web**
- Invented by Tim Berners-Lee at CERN, 1991
- Maintained by the IETF (Internet Engineering Task Force)

- **Every HTTP conversation has two halves:**
  - REQUEST — sent by the browser (client)
  - RESPONSE — sent by the server

HTTP Request — Anatomy

Part	Example	Purpose
Method	GET	What the client wants to do
Path	/menu	Which resource is requested
Version	HTTP/1.1	Protocol version agreement
Headers	Host: site.com	Extra metadata (like order notes)
Body	(optional)	Data sent with POST/PUT

HTTP Methods (Verbs)

Method	Meaning	Analogy
GET	"Give me this resource"	"Can I see the menu?"
POST	"Store this new data"	"Place a new order"
PUT	"Replace entirely"	"Change my whole order"
PATCH	"Update part of it"	"Add jalapeños"
DELETE	"Remove this"	"Cancel my order"

Key Takeaway: Five verbs handle virtually every operation on the web.

HTTP Response — Anatomy

Part	Example	Purpose
Status Code	200 OK	Three-digit outcome summary
Headers	Content-Type: text/html	Metadata about the response
Body	<html>...</html>	The actual content delivered

Status Codes — Success & Redirect

Code	Name	Meaning
200	OK	Everything went fine
201	Created	New resource was created (POST)
301	Moved Permanently	URL changed; update bookmarks
304	Not Modified	Cached version is still valid

- **2xx = Success (green light)**
- **3xx = Redirect (detour sign)**

Status Codes — Client & Server Errors

Code	Name	Meaning
400	Bad Request	Client sent invalid data
401	Unauthorised	Login required
403	Forbidden	Logged in but not allowed
404	Not Found	Resource does not exist
500	Internal Server Error	Server crashed

- **4xx = Client's fault**
- **5xx = Server's fault**

> *Key Takeaway: 2xx good, 3xx redirect, 4xx client error, 5xx server error.*

Lesson 3 — Static Content

- **Files that already exist on disk and are served as-is**
- The server is a file clerk — finds the file, hands it over

- **Examples:**
  - HTML files, CSS stylesheets, JavaScript files
  - Images (PNG, JPG, SVG), fonts, downloadable PDFs

- Analogy: Picking a book off a library shelf — it hasn't changed since last Tuesday

Dynamic Content

- **Generated on the fly for every request**
- Server runs code (Python, PHP, Node.js, Java, C#, etc.)
- Same URL → different result depending on user, time, database

- **Examples:**
  - Your Facebook feed, Amazon search results
  - Bank account balance page, personalised dashboards

- Analogy: Personal stylist curating an outfit based on your taste

Static vs Dynamic — Side by Side

Feature	Static	Dynamic
Speed	Extremely fast	Slower (code must execute)
Personalisation	None	Full (per-user content)
Server load	Minimal	Higher
Example	"About Us" page	Instagram Explore page
Server role	File clerk	Chef cooking to order

Key Takeaway: Most real sites use both — static assets alongside dynamically generated HTML.

Lesson 4 — The Big Three Web Servers

	Apache	Nginx	Node.js
Created	1995	2004	2009
Language	C	C	JavaScript
Architecture	Process-per-request	Event-driven	Event-driven
Typical Role	General-purpose	Reverse proxy / static	App server / API
Config Style	.htaccess / httpd.conf	nginx.conf	JavaScript code

Apache HTTP Server

- **Created 1995 by Apache Software Foundation (ASF)**
- Most popular web server for 20+ years
- Process-per-request model: each visitor gets their own worker
- Configured via .htaccess files — beginner-friendly

- Strengths: Flexibility, massive ecosystem, extensive documentation
- Weakness: Heavy under extreme concurrent load

- Analogy: Sit-down steakhouse — one waiter per table

Nginx (Engine-X)

- **Created 2004 by Igor Sysoev to solve the C10K problem**
  - C10K = handling 10,000 simultaneous connections
- Event-driven, asynchronous architecture
- One process juggles thousands of connections
- Often used as reverse proxy and load balancer

- Analogy: Conveyor-belt sushi — one chef, hundreds of diners

Node.js

- **Created 2009 by Ryan Dahl**
- JavaScript runtime built on Chrome's V8 engine
- Event-driven, non-blocking I/O
- Write your server logic in JavaScript — same language as front-end
- Ideal for real-time apps: chat, streaming, live dashboards
- Managed by OpenJS Foundation

- Analogy: Food truck you build and wire yourself — total flexibility

> *Key Takeaway: Apache = reliable workhorse; Nginx = speed & scale; Node.js = JavaScript everywhere.*

Architecture Comparison

- **APACHE — Process-per-Request:**
  - Request 1 → Worker 1, Request 2 → Worker 2, Request 3 → Worker 3
  - Each request gets its own worker process

- **NGINX — Event Loop:**
  - Request 1, 2, 3 → Single Event Loop → Responses
  - One loop handles all requests asynchronously

- **NODE.JS — Event Loop + Your Code:**
  - Request 1, 2, 3 → V8 Engine + Event Loop + YOUR CODE → Responses
  - Same model, but you write the logic in JavaScript

Lesson 5 — IP Addresses

- **Every computer on a network has a unique IP (Internet Protocol) address**

- IPv4: 192.168.1.10 (4 groups of numbers, 0–255)
- IPv6: 2001:0db8::1 (longer format, vastly more addresses)

- **localhost = 127.0.0.1 = your own machine**
  - No Internet connection needed to reach localhost!

Ports — Apartment Numbers

- **Port numbers (0–65535) distinguish services on the same machine**

- IP address = building's street address
- Port = apartment number inside that building

- http://example.com:8080 means:
  - "Connect to example.com, apartment 8080"

- Managed by IANA (Internet Assigned Numbers Authority)

Well-Known Ports

Port	Protocol / Service
22	SSH (Secure Shell)
80	HTTP (unencrypted web)
443	HTTPS (encrypted web)
3000	Node.js dev server (convention)
3306	MySQL database
5432	PostgreSQL database
27017	MongoDB database

Key Takeaway: Port = apartment number. The mail carrier needs both the address (IP) and apartment (port).

Lesson 6 — Build Your First Web Server

- **What you need:**
  - Node.js installed on your machine
  - A text editor (VS Code, Sublime, etc.)
  - A terminal / command prompt

- **Core pattern:**
  - 1. require('http') — load the HTTP module
  - 2. createServer(callback) — define what happens per request
  - 3. listen(port) — start waiting for connections

Key Takeaway: A working web server can be built in 10 lines of JavaScript. No frameworks needed.

Anatomy of a Minimal Server

- **Four things every server does:**
  - **1. IMPORT the HTTP module**
    - `const http = require('http');`
  - **2. CREATE a server with a handler function**
    - `http.createServer((req, res) => { ... });`
  - **3. SEND a response (status + headers + body)**
    - `res.writeHead(200); res.end('Hello!');`
  - **4. LISTEN on a port**
    - `server.listen(3000);`

The Lemonade Stand Analogy

Lemonade Stand	Web Server
Find a street corner	Bind to an IP and port
Put up a sign with menu	Define your routes / responses
Wait for customers	Call server.listen()
Customer orders	HTTP request arrives
Pour & hand over lemonade	Send HTTP response

Lesson 7 — Routing

- **The mechanism that decides which code runs based on URL + HTTP method**

- Without routing:
  - Every URL → same response (one-room house)

- With routing:
  - / → Home page
  - /about → About page
  - /api/menu → JSON data
  - (other) → 404 Not Found

> *Key Takeaway: URL + method = the address of specific functionality inside your server.*

Routing — The Hotel Concierge

Guest says...	Concierge directs to...	URL equivalent
"Check in please"	Front desk, ground floor	GET /
"Where's the gym?"	Basement level	GET /gym
"Room service!"	Kitchen, 2nd floor	POST /room-service
"Cancel my reservation"	Billing counter	DELETE /reservation/42
"Where's the pool?"	"Sorry, we don't have one"	404 Not Found

Routes Have Two Parts

- **1. PATH — which URL was requested**
  - Examples: / /about /api/users /products/42

- **2. METHOD — which HTTP verb (GET, POST, etc.)**
  - GET /users → list all users
  - POST /users → create a new user
  - Same path, different behaviour!

HTML Routes vs API Routes

	HTML Route	API Route
Returns	HTML for browsers	JSON for apps/code
Content-Type	text/html	application/json
Audience	Humans reading pages	Machines consuming data
Example	/about	/api/users

Key Takeaway: Modern web apps often serve HTML routes AND API routes from the same server.

Lesson 8 — What Is Middleware?

- **A function that sits between the incoming request and your route handler**

- Each middleware can:
  - Read or modify the request object
  - Read or modify the response object
  - End the request early (block it)
  - Pass control to the next middleware in the chain

- Analogy: Airport security checkpoints before your boarding gate

The Middleware Pipeline

- **Request → [Logger] → [Auth] → [Parser] → [Handler] → Response**

- If any checkpoint fails, the request is stopped early
- Order matters — logging before auth, auth before business logic

Checkpoint	What It Does	Airport Analogy
Logger	Records every request	Passport scan records
Authentication	Checks identity	Passport control
Authorisation	Checks permissions	Boarding pass scan
Body Parser	Reads request body	Baggage scanner
Validation	Checks data correctness	Metal detector

Middleware — Why It Matters

- **Middleware is NOT magic — it's just functions running in sequence**

- Every major framework uses this pattern:
  - Express.js (Node.js) — app.use(middleware)
  - Django (Python) — MIDDLEWARE setting
  - Spring (Java) — Filter chain
  - ASP.NET (C#) — Middleware pipeline

- The pattern you learned today transfers to ANY web framework

> *Key Takeaway: Learn the pattern once, apply it everywhere. Middleware is universal.*

Putting It All Together

- **The complete request lifecycle:**
  - 1. Browser crafts an HTTP Request
  - 2. Request travels to server IP + Port
  - 3. Web server software receives it
  - 4. Middleware pipeline processes it (log, auth, parse)
  - 5. Router matches URL + Method to a handler
  - 6. Handler runs your business logic
  - 7. HTTP Response is built (status + headers + body)
  - 8. Response travels back to the browser
  - 9. Browser renders the page

Today's Building Blocks — Checklist

- ✓ Web server concept (hardware + software)
- ✓ HTTP protocol (request + response)
- ✓ Static vs dynamic content
- ✓ Server software: Apache / Nginx / Node.js
- ✓ Ports, IP addresses, and localhost
- ✓ Building a server from scratch in Node.js
- ✓ Routing (URL + method → handler)
- ✓ Middleware (pipeline of functions)

Coming Next Week — Express.js

- **Express.js — the most popular Node.js web framework**

- Express automates everything we built by hand today:
  - Routing: app.get('/about', handler)
  - Middleware: app.use(logger)
  - Static files: express.static('public')
  - JSON parsing: express.json()

- Think of it as upgrading from a hand-built go-kart to a real car

> *Key Takeaway: Today = understand HOW it works. Next week = use Express because we ALREADY understand.*

Key Abbreviations (1/2)

Abbreviation	Full Form
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
API	Application Programming Interface
URL	Uniform Resource Locator
IP	Internet Protocol
JSON	JavaScript Object Notation

Key Abbreviations (2/2)

Abbreviation	Full Form
DNS	Domain Name System
TCP	Transmission Control Protocol
SSH	Secure Shell
FTP	File Transfer Protocol
SMTP	Simple Mail Transfer Protocol
IANA	Internet Assigned Numbers Authority
IETF	Internet Engineering Task Force
CERN	European Organization for Nuclear Research

Resources & Further Reading

- IETF — https://www.ietf.org/
- MDN HTTP Reference — https://developer.mozilla.org/en-US/docs/Web/HTTP
- Node.js — https://nodejs.org/
- Express.js — https://expressjs.com/
- Apache — https://httpd.apache.org/
- Nginx — https://www.nginx.com/
- RFC 9110 (HTTP Spec) — https://www.rfc-editor.org/rfc/rfc9110.html

- Always verify URLs at: https://www.virustotal.com/gui/home/url

Disclaimer

- **© 2026 Cyber.SoHo Educational Hub | Schiller International University**
- All rights reserved.

- This material is not affiliated with, endorsed by, or sponsored by any
- organisation mentioned herein (Apache, Nginx, Node.js, Google, etc.).

- External links are provided for educational convenience only.
- Verify all URLs at https://www.virustotal.com/gui/home/url

- Provided "as-is" for educational purposes. No liability assumed.
- Code examples are for learning — review before production use.